

# 04 - Backend / API Denetimi

Olusturulma: 14 Nisan 2026 19:49 Kaynak: docs/audit Format: Tekil rapor

## 04 - Backend / API Denetimi

Denetim tarihi: 2026-04-14 Kapsam: order lifecycle, table lifecycle, payment rules, kitchen/print trigger mantigi, masa tasima, API guard'lari, status transition'lar, duplicate action riskleri, transaction ve veri tutarililigi.

Bu rapor yalnızca analizdir. Uygulama kodunda davranissal degisiklik yapilmamistir.

### 1. Backend mimarisi ozeti

Backend Express tabanlı, SQLite/better-sqlite3 üzerinden senkron transaction modeli kullanan bir POS API katmanı olarak çalışıyor. Ana domain yüzeyi şu dosyalarda toplanmış durumda:

- `server/index.js` : API route montajlari. Tables `server/index.js:113` , orders `server/index.js:116` , payments `server/index.js:117` , admin `server/index.js:123` , bridge `server/index.js:124` .
- `server/routes/orders.js` : siparis olusturma, urun ekleme, paket teslim, mutfak status gecisleri, satir guncelleme, kapatma/iptal.
- `server/routes/payments.js` : tam odeme, parcali/split odeme, odeme ozeti, odeme ile siparis/masa kapatma.
- `server/routes/tables.js` : masa listesi, masa status guncelleme, masa transferi.
- `server/services/printJobs.js` : mutfak, fis, paket etiketi ve yazici job idempotency mantigi.
- `server/routes/bridge.js` : StoreBridge/yazici istemcisinin job listeleme, claim etme ve sonuc bildirme API'leri.
- `server/migrations/run.js` : temel tablo semalari, enum/check constraint'leri ve migration yamalari.

Guclu taraflar:

- Kimlik dogrulama ve business scope middleware'i merkezi kurulmus; route'lar `req.businessId` ile isletme izolasyonu uyguluyor.
- Siparis olusturma, urun ekleme, masa transferi ve odeme alma gibi kritik islemlerin onemli bolumlerinde SQLite transaction kullaniliyor.
- Yazici job tarafinda `idempotency_key` UNIQUE constraint'i var ( `server/migrations/run.js:297-299` ) ve job insert'i `INSERT OR IGNORE` ile yapiliyor ( `server/services/printJobs.js:63-79` ).
- Bridge claim islemi atomik: sadece `pending` ve `claimed_at IS NULL` olan job claim ediliyor ( `server/routes/bridge.js:329-342` ).
- Son degisikliklerden sonra paket teslim akisi backend tarafinda otomatik odeme kaydi ile kapanacak sekilde desteklenmis ( `server/routes/orders.js:214-225` ).

Ana zayiflik: backend, domain kurallarini tek bir servis/domain katmaninda toplamiyor. Kurallar route icinde daginik: status gecisleri, odeme guard'lari, masa status kurallari, mutfak/yazici side effect'leri ayni dosyalarda

karisik yurutuluyor. Bu, bugunku olcekte calisir; fakat restoran POS gibi hizli ve tekrara acik ortamda duplicate action, eksik transaction ve UI'a guvenme riskini buyutur.

## 2. Kritik domain akislar

### 2.1 Masa siparisi olusturma

1. `POST /api/orders` create schema ile validate edilir ( `server/routes/orders.js:62-75` ).
2. Request icinde en az bir urun zorunludur ( `server/routes/orders.js:367-369` ).
3. Transaction icinde sira numarasi alinir, urun fiyatları çözülür, order `saved` status ile eklenir ( `server/routes/orders.js:374-421` ).
4. Order item'lar `new` status ile eklenir ( `server/routes/orders.js:423-433` ).
5. `table_id` varsa masa `occupied` yapilir ve `current_order_id` atanir ( `server/routes/orders.js:436-438` ).
6. Transaction bittikten sonra paket etiketi, Caller ID link'i ve mutfak finalize islemleri calisir ( `server/routes/orders.js:442-452` ).
7. `finalizeKitchenForNewItems` yeni satirlari `sent`, siparisi `in_kitchen` yapar ve mutfak job tetikler ( `server/routes/orders.js:87-95` ).

Risk: DB kaydi transaction icinde, mutfak/yazici/caller side effect'leri transaction disinda. Bu ayrim, siparis kaydedildikten sonra mutfak job/status finalize basarisiz olursa siparisin kullaniciya kaydedilmis gorunup mutfak tarafina eksik dusmesine yol acabilir.

### 2.2 Kayitli siparise yeni urun ekleme

1. `POST /api/orders/:id/items` addItems schema ile en az bir urun ister ( `server/routes/orders.js:77-80`, `server/routes/orders.js:470` ).
2. Kapali veya iptal edilmiş siparise urun ekleme engellenir ( `server/routes/orders.js:479-480` ).
3. Transaction icinde urunler eklenir ve toplamalar guncellenir.
4. Transaction sonrasi yeni item'lar mutfaga finalize edilir ( `server/routes/orders.js:521` ).

Risk: Olusturma akisina benzer sekilde yeni satirlar DB'ye yazildiktan sonra mutfak status/job islemi ayrik calisir. Bu, "urun eklendi ama mutfaga gitmedi" veya "satir new kaldı" gibi POS icin yuksek maliyetli operasyonel tutarsizliklara acik.

### 2.3 Siparis kapatma

1. `PATCH /api/orders/:id/status` status body alir ( `server/routes/orders.js:536-539` ).
2. `cancelled` icin yetki, mevcut status ve odeme varligi kontrol edilir ( `server/routes/orders.js:542-553` ).
3. `closed` icin artik backend odeme tamamlanmadan kapatmayi engelliyor ( `server/routes/orders.js:584-586` ).
4. `closed` oldugunda masa bosaltilir ve musterii istatistigi guncellenir ( `server/routes/orders.js:592-607` ).
5. Dine-in sipariste fis job'u transaction sonrasi enqueue edilir ( `server/routes/orders.js:617-619` ).

Guc: Odeme tamamlanmadan `closed` status engeli backend tarafinda var. Bu, onceki urun/is kurallari raporlarında P1 olarak isaretlenen en kritik davranislardan birinin dogru yere alindigini gosteriyor.

Risk: `status` request body icin zod enum validation yok. Gecersiz status, DB CHECK constraint'e carpip 500'e donebilir. Profesyonel API'de bu 400 ve domain error code olmalidir.

## 2.4 Paket siparis teslim akisi

1. `PATCH /api/orders/:id/takeaway/delivery` yalnızca `out_for_delivery` ve `delivered` aksiyonlarını kabul eder ( `server/routes/orders.js:176-180` ).
2. Sadece `takeaway` order type guncellenebilir ( `server/routes/orders.js:187-188` ).
3. `out_for_delivery` birden fazla çağrılırsa idempotent olarak `already_out_for_delivery` döner ( `server/routes/orders.js:199-205` ).
4. `delivered` için önce `takeaway_out_at` zorunludur ( `server/routes/orders.js:208-209` ).
5. Delivery transaction'ini içinde kalan tutar kadar otomatik ödeme kaydedilir ve order `closed` yapılır ( `server/routes/orders.js:214-221` ).

Guc: Kullanıcının son talebiyle uyumlu olarak, paket siparis teslim edilince ödeme zorunluluğu kaldırılmış ve backend kalan tutarı otomatik ödendi olarak isliyor.

Risk: Endpoint önce `closed/cancelled` kontrolü yapıyor ( `server/routes/orders.js:190-192` ), sonra `takeaway_delivered_at` idempotency kontrolüne geliyor ( `server/routes/orders.js:193-197` ). Başarılı teslimden sonra aynı `delivered` aksiyonu tekrar gelirse order status `closed` olduğu için idempotent cevap yerine 400 dönebilir. Veri bozulmaz; fakat POS'ta çift tıklama/ag kopması sonrası kullanıcıya hatalı hata mesajı üretir.

## 2.5 Odeme alma

1. `POST /api/payments` sadece `cash` ve `card` kabul eder ( `server/routes/payments.js:13-23` ).
2. Kapalı/iptal siparise ödeme eklenemez ( `server/routes/payments.js:203-207` ).
3. Transaction içinde payment kaydı eklenir ( `server/routes/payments.js:214-218` ).
4. `close_order` true ise ödeme yeterliyse order ve masa kapatılır ( `server/routes/payments.js:220-222` ).
5. Receipt job transaction sonrası tetiklenir.

Risk: Normal odemede kalan bakiye asimi kontrolü yok. Split payment tarafında bu guard var ( `server/routes/payments.js:315-329` ), fakat tam ödeme endpoint'i `amount > remaining` durumunu engellemiyor. Bu, grand\_total üzerinde paid\_total oluşmasına ve raporlamada sahte ciro görünmesine yol açabilir.

## 2.6 Parcali / split ödeme

1. `POST /api/payments/:orderId/split` item allocation üzerinden miktar hesaplar.
2. Daha önce unallocated full-order payment varsa split'i engeller ( `server/routes/payments.js:159` , `server/routes/payments.js:263-264` civarı).
3. Kalan tutar asimi engellenir ( `server/routes/payments.js:315-329` ).
4. Test davranışına göre tüm split ödemeler tamamlanınca order otomatik kapanmıyor; masa/siparis explicit kapanış bekliyor.

Yorum: Bu davranış ürün kararı olabilir. Ancak backend'de "paid but open" gibi acik bir status yok. Bu durum UI'da ve raporlarda net ayrılmazsa personel "odendi mi, kapanacak mi" belirsizliği yasar.

## 2.7 Masa tasima

1. `POST /api/tables/:id/transfer` target zorunlu, kaynak/hedef aynı olamaz ( `server/routes/tables.js:86-94` ).
2. Kaynak ve hedef masa aynı business içinde aranır ( `server/routes/tables.js:95-98` ).

3. Hedef masa sadece `empty` ise kabul edilir; reserved hedefe transfer engellenir ( `server/routes/tables.js:99-100` ).
4. Transaction icinde order `table_id` hedefe tasinir, hedef masa kaynak state'ini alır, kaynak masa bosaltılır ( `server/routes/tables.js:103-110` ).

Guc: Rezerve masaya transfer/acma kurali backend'de guclu bir guard ile korunuyor.

Risk: Generic `PATCH /api/tables/:id/status` ayni domain guvenligini tasimiyor. Herhangi bir masa status'u, current order invariant'lari kontrol edilmeden degistirilebilir ( `server/routes/tables.js:59-73` ).

## 2.8 Yazici / mutfak akisi

1. Mutfak, fis ve paket etiketi job'lari `print_jobs` tablosuna yaziliyor.
2. Job status enumlari: `pending` , `printed` , `failed` , `cancelled` ( `server/migrations/run.js:290-299` ).
3. Idempotency key unique ve insert ignore ile duplicate job azaltilmis ( `server/services/printJobs.js:63-79` ).
4. Bridge pending joblari listeler ve atomik claim eder ( `server/routes/bridge.js:300-342` ).
5. Bridge job sonucunu `printed` veya `failed` olarak bildirir ( `server/routes/bridge.js:357-376` ).

Risk: `processPendingJobsSync` mock modu kapali degilse pending joblari server tarafinda otomatik `printed` yapabiliyor ( `server/services/printJobs.js:592-610` ). Production konfigrasyonunda bu kritik bir operasyonel toggle. Yanlis config ile gercek yaziciya gitmeyen job "printed" gorunebilir.

## 3. Status transition matrisi

### 3.1 Order

| Status     | Olusturan akis                                                                                                                                                | Izin verilen temel sonraki durumlar                                                                                    | Backend guard durumu                                                          |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| new        | DB default<br>( <code>server/migrations/run.js:198</code> )                                                                                                   | Pratikte nadir; order create<br><code>saved</code> yazar                                                               | Domain'de aktif kullanimi belirsiz                                            |
| saved      | Order create<br>( <code>server/routes/orders.js:413-421</code> )                                                                                              | <code>in_kitchen</code> , <code>cancelled</code> ,<br><code>closed</code>                                              | Create sonrasi otomatik finalize genelde <code>in_kitchen</code> yapar        |
| in_kitchen | <code>finalizeKitchenForNewItems</code> veya status patch<br>( <code>server/routes/orders.js:87-95</code> ,<br><code>server/routes/orders.js:575-615</code> ) | <code>preparing</code> , <code>ready</code> ,<br><code>served</code> , <code>cancelled</code> ,<br><code>closed</code> | Status patch enum validation eksik                                            |
| preparing  | Status veya item tarafından operasyonel ilerleme                                                                                                              | <code>ready</code> , <code>served</code> ,<br><code>closed</code> , <code>cancelled</code>                             | Gecis kurallari merkezi degil                                                 |
| ready      | Mutfak/satir ilerleme                                                                                                                                         | <code>served</code> , <code>closed</code>                                                                              | Merkezi transition graph yok                                                  |
| served     | Servis edildi bilgisi                                                                                                                                         | <code>closed</code>                                                                                                    | Closed icin odeme guard'i var                                                 |
| cancelled  | Status patch veya auto-cancel                                                                                                                                 | Terminal                                                                                                               | Odeme varsa iptal engeli var                                                  |
| closed     | Odeme ile kapatma, status patch, paket delivered                                                                                                              | Terminal                                                                                                               | Odeme tamamlanmadan closed engeli var; paket delivered otomatik odeme yaratir |

### 3.2 Order item

| Status    | Olusturan akis                                       | Beklenen sonraki durum                    | Risk                                                                             |
|-----------|------------------------------------------------------|-------------------------------------------|----------------------------------------------------------------------------------|
| new       | Item insert<br>( server/migrations/run.js:226 )      | sent , cancelled ,<br>porsiyon degisimi   | Porsiyon degisimi sadece new<br>iken engelleniyor/destekleniyor                  |
| sent      | Mutfak finalize<br>( server/routes/orders.js:90-92 ) | preparing , ready ,<br>served , cancelled | Satir status'u request body'den<br>enum validation olmadan<br>gelebiliyor        |
| preparing | Item patch<br>( server/routes/orders.js:657-662 )    | ready , cancelled                         | Gecis sirasi kontrolu yok                                                        |
| ready     | Item patch/kitchen                                   | served , cancelled                        | Merkezi kural yok                                                                |
| served    | Item patch                                           | Terminal veya cancelled ?                 | Terminal mantigi net degil                                                       |
| cancelled | Item patch/order cancel                              | Terminal                                  | Cancel sonrasi tekrar status<br>degisimi guard'i zayif                           |
| comped    | DB enum<br>( server/migrations/run.js:226 )          | Terminal/raporlama                        | Kod daha cok is_comped<br>flag'i kullaniyor; enum ile flag<br>cift anlam tasiyor |

### 3.3 Table

| Status   | Olusturan akis                                                                                                                                               | Anlam           | Risk                                                                         |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|------------------------------------------------------------------------------|
| empty    | Default, order close/cancel,<br>transfer source<br>( server/migrations/run.js:80 ,<br>server/routes/orders.js:598-602 ,<br>server/routes/tables.js:109-110 ) | Aktif order yok | Generic status patch ile<br>current_order_id temizlenmeden<br>set edilebilir |
| occupied | Order create, transfer target<br>( server/routes/orders.js:436-438 ,<br>server/routes/tables.js:107-108 )                                                    | Aktif order var | Generic patch ile order olmadan<br>occupied olabilir                         |
| reserved | Table status patch/admin akisi                                                                                                                               | Rezerve masa    | Transfer target olarak<br>engelleniyor<br>( server/routes/tables.js:99-100 ) |

### 3.4 Payment

Payment icin status kolonu yok; kayıt tipi ve scope ile durum turetiliyor.

| Alan          | Degerler                      | Kaynak                          |
|---------------|-------------------------------|---------------------------------|
| payment_type  | DB: cash, card, mixed, other  | server/migrations/run.js:242    |
| payment_type  | API create: sadece cash, card | server/routes/payments.js:13-18 |
| payment_scope | full_order, split_item        | server/migrations/run.js:243    |

Not: Paket teslim otomatik odemesi backend icinde `other` olarak kaydediliyor. Bu dogru bir teknik isaret olabilir; fakat rapor/finans UI tarafinda "otomatik teslim odemesi" olarak ayrismali.

### 3.5 Kitchen / print job

| Status    | Olusturan akis               | Anlam               | Risk                                               |
|-----------|------------------------------|---------------------|----------------------------------------------------|
| pending   | Job enqueue                  | Yazdirilacak is     | Bridge veya mock tarafından islenir                |
| printed   | Bridge PATCH veya mock       | Basarili yazdirildi | Mock production'da acik kalirsa sahte basari riski |
| failed    | Bridge PATCH veya mock catch | Yazdirma basarisiz  | Retry/resolve workflow'u sinirli gorunuyor         |
| cancelled | DB enum                      | lptal is            | Aktif endpoint/akis net degil                      |

## 4. API guard ve is kurali eksikleri

### P1 - API schema ile DB order\_type constraint'i uyumsuz

- Kanit: API `order_type` icin `delivery` kabul ediyor ( `server/routes/orders.js:64` ), fakat DB sadece `dine_in` ve `takeaway` kabul ediyor ( `server/migrations/run.js:194` ).
- Etki: Client `delivery` gonderirse validation gecer, insert DB CHECK constraint'e takilir ve profesyonel 400 yerine 500/teknik hata uretme riski olusur.
- Kok neden: API contract ile migration enum'u ayni kaynaktan uretilmiyor.
- Oneri: V1'de `delivery` henuz gercek domain tipi degilse API enumundan kaldir. Eger delivery ayri tip olacaksa migration, UI, raporlar, paket teslim endpoint'i ve testler birlikte guncellenmeli.

### P1 - Normal odeme endpoint'i kalan bakiye asimini engellemiyor

- Kanit: `POST /api/payments` amount'u positive olarak validate ediyor ( `server/routes/payments.js:19` ) ve direkt insert ediyor ( `server/routes/payments.js:214-218` ). Split payment'ta kalan tutar asimi guard'i var ( `server/routes/payments.js:315-329` ), normal odemede yok.
- Etki: Ayni siparise fazla odeme yazilabilir. Gun sonu raporu, masa odenmislik durumu ve finansal toplamalar `grand_total`'dan yuksek gorunebilir.
- Kok neden: Full payment ile split payment ayni payment policy helper'ini kullanmiyor.
- Oneri: `remaining_total` merkezi hesaplanmeli; tip/bahsis modeli yoksa `amount <= remaining_total + tolerans` guard'i full payment icin de uygulanmeli. Bahsis desteklenecekse ayrica tipalani ve raporlama ayrimi eklenmeli.

## P1 - Odeme endpoint'lerinde idempotency yok

- Kanit: `POST /api/payments` her cagrida yeni `paymentId` uretip insert ediyor (`server/routes/payments.js:209-218`). Request idempotency key veya unique client operation id yok.
- Etki: Kasiyer cift tiklar, ag timeout sonrasi tekrar dener veya POS tablet istegi yinelerse ayni odeme iki kez kaydedilebilir. Bu restoran operasyonunda dogrudan kasa/fis/rapor problemi yaratir.
- Kok neden: Payment write API'si "at least once" client davranisina karsi korunmuyor.
- Oneri: `Idempotency-Key` veya body'de `client_operation_id` kabul edilmeli; `payments` tablosunda `business/order/key` unique constraint ile tekrar cagrida mevcut payment donmeli.

## P1 - Order item update backend'i UI'a fazla guveniyor

- Kanit: Item patch body'den `status`, `quantity`, `is_comped`, `comp_reason` aliniyor (`server/routes/orders.js:632-634`), `status` ve `quantity` dogrudan SQL update'e giriyor (`server/routes/orders.js:655-690`). Quantity icin positive/int guard'i, status icin enum/transition validation yok.
- Etki: Negatif veya sifir quantity, gecersiz status, terminal satirin tekrar degismesi, comp flag/status uyumsuzlugu gibi veri bozan durumlar API seviyesinde mumkun hale gelir.
- Kok neden: UI tarafindaki kontroller domain guard yerine gecmis.
- Oneri: `patchOrderItemSchema` eklenmeli; quantity integer/positive/max olmalı, status enum ve izinli gecis tablosu ile kontrol edilmeli, `is_comped` ile `status='comped'` kararından biri tek kaynak yapılmalı.

## P1 - Siparis create/add-items akisinda side effect'ler transaction disinda

- Kanit: Order transaction `txn()` ile biter (`server/routes/orders.js:442`), sonra paket etiketi, Caller ID link'i ve mutfak finalize calisir (`server/routes/orders.js:444-452`). Add-items da transaction sonrasi `finalizeKitchenForNewItems` cagirir (`server/routes/orders.js:521`).
- Etki: Siparis DB'de kayitli kalip mutfak status'u/job'u eksik kalabilir. Personel "kaydetti", mutfak "gormedi" senaryosu restoran icin kritik operasyonel risktir.
- Kok neden: Write model ve outbox/side effect modeli ayrilmamis.
- Oneri: Order/item status guncellemeleri ana transaction icinde yapilmali. Yazici/caller side effect'leri icin transaction icinde outbox/job kaydi olusturulmalı, harici isleme sonradan guvenli retry ile calismali.

## P2 - Generic table status endpoint invariant bozabilir

- Kanit: `PATCH /api/tables/:id/status`, status'u `current_order_id` ile tutarlilik kontrolu yapmadan update ediyor (`server/routes/tables.js:59-73`).
- Etki: Masa `occupied` ama `current_order_id` null, veya masa `empty` ama aktif order masaya bagli gibi UI ve raporlamayi bozan durumlar olusabilir.
- Kok neden: Masa status'u domain action yerine serbest alan guncellemesi gibi modellenmis.
- Oneri: Generic status patch kisitlanmalı. `reserveTable`, `clearReservation`, `seatGuests`, `closeTable` gibi domain endpoint'leri ya da en azindan status bazli invariant guard'lari getirilmeli.

## P2 - Delivered retry idempotency sirasinda kapali siparis erken 400 donuyor

- Kanit: Paket delivery endpoint'i once `closed/cancelled` kontrolu yapar (`server/routes/orders.js:190-192`), sonra `takeaway_delivered_at` idempotency cevabina bakar (`server/routes/orders.js:193-197`).



- Etki: Basarili teslimden sonra ayni istek tekrar gelirse kullanıcı "Siparis kapali" hatasi alabilir. Veri bozulmaz; operasyonel guven hissi zedelenir.
- Kok neden: Terminal status guard'i idempotent sonuc kontrolunden once calisiyor.
- Oneri: `takeaway_delivered_at && action === 'delivered'` kontrolu closed guard'dan once ele alinmali.

## P2 - Payment type kontrati iki farkli yerde farkli

- Kanit: DB `cash`, `card`, `mixed`, `other` kabul ediyor ( `server/migrations/run.js:242` ), public payment API sadece `cash`, `card` kabul ediyor ( `server/routes/payments.js:16-18` ), paket otomatik odeme ise backend icinde `other` uretiyor.
- Etki: Raporlama, UI etiketi ve API dokumantasyonunda "other" odemenin anlami belirsiz kalabilir. Otomatik paket kapanislarinda ciro tipi yanlis yorumlanabilir.
- Kok neden: Payment type semantigi domain seviyesinde belgelenmemis; API ve internal flow ayrimi net degil.
- Oneri: `other` sadece sistem kaynakli odeme ise `source='system_takeaway_delivery'` gibi ayri alan eklenmeli veya note/source ile raporlarda ayrismali. Public API enum'u ve DB enum'u bilincli olarak dokumante edilmeli.

## 5. Transaction ve veri tutariligi riskleri

### P1 - Item update + totals recalculation atomik degil

- Kanit: Item update yapiliyor ( `server/routes/orders.js:688-690` ), ardindan `recalcOrderTotals` ve `autoCancelOrderIfNoActiveItems` transaction disinda ayri cagriliyor ( `server/routes/orders.js:693-694` ).
- Etki: Update basarili, toplam recalculation basarisiz olursa order total eski kalabilir. Cancel edilen son urun sonrasi order auto-cancel eksik kalabilir.
- Kok neden: "Satir guncelle + toplam hesapla + terminal durum kontrolu" tek domain command degil.
- Oneri: Bu uc adim tek transaction icine alinmali; failure halinde tum islem rollback olmalı.

### P1 - Payment double submit finansal veri tutariligini bozar

- Kanit: Payment insert tekil request kimligiyle korunmuyor ( `server/routes/payments.js:209-218` ).
- Etki: Iki payment kaydi, iki audit, muhtemelen iki fis/job. Geri alma veya muhasebe duzeltmesi gerekir.
- Kok neden: API idempotency tasarimi eksik.
- Oneri: Payment ve split payment endpoint'leri icin idempotency zorunlu hale getirilmeli.

### P1 - Siparis kaydi ile mutfak/yazici job garanti modeli zayif

- Kanit: `txn()` bittikten sonra `enqueueTakeawayLabelJob`, `linkCallLogToOrder`, `finalizeKitchenForNewItems` calisiyor ( `server/routes/orders.js:442-452` ).
- Etki: Siparis kayitli ama mutfak job'u yok; Caller ID order'a baglanmamis; paket etiketi basilmamis.
- Kok neden: Transactional outbox pattern yok.
- Oneri: DB transaction icinde islenecek job/outbox kaydi olusturulmali. Harici islem retry edilebilir worker/bridge tarafina birakilmali.

## P2 - Print mock production'da yanlış config ile sahte başarı üretebilir

- Kanıt: `processPendingJobsSync` config kapalı değilse pending jobları otomatik `printed` yapıyor (`server/services/printJobs.js:592-610`).
- Etki: Yazıcıya gitmeyen adisyon/mutfak fişi printed görünebilir.
- Kök neden: Test/dev kolaylığı ile production davranışı aynı servis içinde.
- Öneri: Production'da mock processor hard-disabled olmalı; config startup'ta loglanmalı ve env validation ile güvenceye alınmalı.

## P2 - Status geçişleri merkezi değil

- Kanıt: Order status, item status, table status farklı route bloklarında if/else ile yönetiliyor (`server/routes/orders.js:536-620`, `server/routes/orders.js:632-696`, `server/routes/tables.js:59-73`).
- Etki: Bir ekranda engellenen geçiş başka endpoint'te mümkün olabilir. Test kapsamı büyüdükçe kurallar kopyalanır.
- Kök neden: Domain transition matrix kodda tek kaynak değil.
- Öneri: `domain/orderLifecycle`, `domain/paymentPolicy`, `domain/tablePolicy` gibi saf fonksiyonlardan oluşan merkezi guard katmanı kurulmalıdır.

## 6. Amatör görünen backend alanları

### P1 - Route dosyaları domain servis gibi davranmaya başlamış

`server/routes/orders.js` hem validation, hem fiyat çözme, hem DB write, hem mutfak/yazıcı side effect, hem audit, hem socket emit yapıyor. Bu dosya restoran POS'un en kritik domainini taşıyor ama route handler seviyesinde buyumuş. Bu profesyonel ürün kalitesinde bakım riskidir.

Öneri: Önce düşük riskli olarak saf policy/helper dosyaları çıkarılmalı; sonra route handler'lar "request -> command -> response" seviyesine indirilmeli.

### P1 - Backend hata modeli tutarsız ve tanı koydurumuyor

Birçok catch bloğu `Sunucu hatası` donuyor; bazı business hataları plain Turkish string, bazı teknik hatalar DB constraint kaynaklı 500 olabilir. Örnek: tables update catch'i sadece generic 500 doner (`server/routes/tables.js:79-81`). Bridge tarafında daha iyi `claim_failed` gibi kodlu hata var (`server/routes/bridge.js:339-342`).

Öneri: `{ code, message, details?, correlation_id }` formatına geçilmeli. Kullanıcı mesajı ile teknik hata kodu ayrılmalı.

### P1 - UI'a güvenen validation kalıntıları var

Order item patch, full payment overpay ve table status patch bu kategoride. UI bug'i veya manuel API çağırışı direkt veri kalitesine zarar verebilir.

Öneri: Frontend hiç yokmuş gibi backend domain guard yazılmalı.

## P2 - Payment type semantigi urun diliyle net degil

`other` otomatik paket teslim odemesi icin kullaniliyor, DB `mixed` destekliyor, public API sadece `cash/card` kabul ediyor. Bu, ileride rapor ekranlarinda "diger odeme" siserek isletmecinin nakit/kart ciro takibini bozar.

Oneri: Otomatik teslim kapatma icin ayri source alanlari ve rapor etiketi eklenmeli.

## P2 - Mock printer log'u ve Unicode ikonlu console ciktilari backend servisinde duruyor

`processPendingJobsSync` icinde console log var (`server/services/printJobs.js:606-608`). Bu dev/test icin faydali olabilir; fakat production log standardi, structured logging ve encoding acisindan profesyonel degil.

Oneri: Structured logger kullanilmali; mock path sadece test/dev konfigrasyonunda aktif olmalı.

# 7. Hemen ele alınmasi gerekenler

## P1 - Payment idempotency

Odeme endpoint'leri duplicate submit'e karsi korunmalı. Bu, finansal veri tutarlıligi icin en yuksek oncelikli backend hardening maddesidir.

Minimum profesyonel cozum:

- `payments` tablosuna nullable `idempotency_key` ve `source` alanlari ekle.
- `business_id + order_id + idempotency_key` unique index ekle.
- Ayni key tekrar gelirse yeni payment yaratma; mevcut payment'i don.
- Split payment icin allocation hash veya client operation id zorunlu yap.

## P1 - Full payment remaining guard

Normal odemede kalan tutar asimi engellenmeli veya tip/bahsis modeli acikca ayrilmali.

Minimum cozum:

- `getPaymentTotals(order_id)` helper'i full payment endpoint'inde kullan.
- `amount > remaining_total + 0.02` ise 400 don.
- Hata kodu: `PAYMENT_AMOUNT_EXCEEDS_REMAINING`.

## P1 - Order/item validation

Order item patch icin zod schema ve transition guard eklenmeli.

Minimum cozum:

- `quantity`: int, min 1, max makul limit.
- `status`: enum ve izinli gecis.
- `is_comped` ve `status='comped'` icin tek kaynak karari.
- Satir degisimi + total recalculation tek transaction.

## P1 - API/DB order\_type uyumu

`delivery` kabul edilecekse DB ve domain tam desteklemeli; edilmeyecekse schema'dan kaldirilmali.

V1 icin daha guvenli secenek: API enumunu `dine_in | takeaway` ile sinirlamak.

## P1 - Transactional outbox / mutfak finalize siniri

Siparis kaydi ile mutfak/yazici/caller side effect'leri arasindaki bosluk kapatilmali.

Minimum cozum:

- Order/item status finalize DB transaction icine alinmali.
- Print job insert'leri transaction icinde outbox kaydi olarak garanti edilmeli.
- Harici yazdirma islemi retry edilebilir kalmali.

## 8. Sonraki turda uygulanabilecek dusuk riskli duzeltmeler

### 1. Status endpoint validation'lari

- `PATCH /api/orders/:id/status` icin zod enum.
- `PATCH /api/orders/:orderId/items/:itemId` icin zod schema.
- `PATCH /api/tables/:id/status` icin enum + invariant guard.

Risk: Dusuk/orta. Client'in gecersiz request gonderdigi gizli yerler varsa ortaya cikar; bu iyi bir kirilma olur.

### 2. Paket delivered retry cevabini idempotent hale getirme

- `takeaway_delivered_at && action === 'delivered'` kontrolunu closed guard'dan once ele al.

Risk: Dusuk. Veri davranisi degismez, tekrar cagrida daha dogru API cevabi doner.

### 3. Full payment overpay guard

- Split payment'taki kalan bakiye guard mantigi full payment endpoint'ine tasinir.

Risk: Orta. Mevcut kullanimda bilincli fazla odeme giriliyorsa engellenir; fakat tip modeli olmadigi icin bu daha dogru davranistir.

### 4. Payment source etiketi

- Otomatik paket teslim odemelerine `source` / `note` standardi ver.
- Raporlarda "Paket teslim otomatik kapanis" olarak ayir.

Risk: Dusuk/orta. Migration ve rapor UI etkisi var.

### 5. Print mock env guard

- Production ortaminda `disablePrintJobMock=true` zorunlu hale getir.
- Startup validation ile aksi halde uygulamayi baslatma veya sert uyari ver.

Risk: Dusuk. Yazici entegrasyon testleri icin env ayrimi netlesir.

### 6. Route dosyalarindan saf domain helper'lari cikarma

Ilk refactor parcalari dusuk riskli olabilir:

- `paymentPolicy.js` : remaining total, close eligibility, overpay guard.
- `orderLifecycle.js` : status transition guard'lari.

- `tablePolicy.js`: table invariant guard'lari.
- `printOutboxPolicy.js`: job idempotency helper'lari.

Risk: Orta. Davranis degistirmeden test esliginde tasinmali.

## Ozet onceliklendirme

### P0

Bu turda kanitli P0 bulgu yok. Uygulamada finansal ve operasyonel P1 riskler var; fakat anlik veri kaybi veya tum sistemi durduran kanitli bir P0 tespit edilmedi.

### P1

1. Odeme endpoint'lerinde idempotency yok.
2. Normal odeme kalan bakiye asimini engellemiyor.
3. Order item patch validation ve transition guard eksik.
4. API `delivery` order type kabul ediyor, DB etmiyor.
5. Siparis kaydi ile mutfak/yazici/caller side effect'leri atomik garanti altinda degil.
6. Item update + total recalculation transaction icinde degil.

### P2

1. Generic table status endpoint masa invariant'larini bozabilir.
2. Paket delivered retry idempotency sirasi kullaniciya hatali 400 dondurebilir.
3. Payment type/source semantigi raporlama icin belirsiz.
4. Print mock production config riski tasiyor.
5. Status transition'lar merkezi degil.
6. Hata modeli kodlu, tani koydurabilir ve tutarli degil.

## Son karar

Backend bugunku haliyle temel POS akislarini tasiyor ve son turdaki odeme/masa/paket guard'lari urun risklerini azaltti. Ancak profesyonel urun kalitesi icin en kritik eksik, backend'in "UI dogru davranir" varsayimini tamamen terk etmemis olmasi. Odeme idempotency, kalan bakiye guard'i, item validation ve transaction sinirlari cozulmeden sistem yogun restoran operasyonunda hatayi kullanıcı aliskanligina, ag kosullarina ve hizli dokunmatik kullanimina fazla acik birakiyor.